

BCSE335L- Healthcare Data Analytics

COURSE PROJECT WORK - REPORT

Title: Feature Engineering and Optimised Machine Learning Methods for Heart Disease Prediction

TEAM MEMBERS:

21BDS0285-Mukkamalla Tarunika

22BCE2183- Angalagunta Shravani

22BCE2299- L. Ashwitha

22BCE2306- Kadapa Azeez

Introduction:

Heart diseases are the leading causes of death around the world, and thus their early diagnosis and effective treatment are of prime importance. Many traditional detection methods are very time-consuming and heavily reliant on clinical expertise. Machine learning can emerge as a promising alternative by identifying hidden patterns in medical data to predict diseases with high accuracy and speed.

This introduces a machine learning-based framework for the prediction of heart disease using the Kaggle Heart Disease dataset. The 25 erroneous records are cleaned, and the features are engineered using techniques such as StandardScaler. And also tested with different types of machine learning algorithms such as Logistic Regression,XG Boost,SVM.

Dataset Description:

The dataset used in this study is the Cleveland Heart Disease dataset from Kaggle. It consists of 303 records and 14 attributes, each representing patient medical parameters used to predict the presence and severity of heart disease. The dataset contains no missing values, and further preprocessing resulted in removing 25 erroneous entries to provide more reliable and consistent data for machine learning analysis.

Attribute Overview:

Feature	Description	Data Type	Range / Values
age	Age of the patient in years	Numeric	29 – 77
sex	Gender (1 = Male, 0 = Female)	Binary	0, 1
cp	Chest pain type (0–3: typical angina to asymptomatic)	Categorical	0, 1, 2, 3
trestbps	Resting blood pressure (mm Hg)	Numeric	94 – 200
chol	Serum cholesterol level (mg/dl)	Numeric	126 – 564
fbs	Fasting blood sugar >120 mg/dl (1 = True, 0 = False)	Binary	0, 1

Feature	Description	Data Type	Range / Values
restecg	Resting electrocardiographic results (0–2)	Categorical	0, 1, 2
thalach	Maximum heart rate achieved	Numeric	71 – 202
exang	Exercise-induced angina (1 = Yes, 0 = No)	Binary	0, 1
oldpeak	ST depression induced by exercise relative to rest	Numeric	0.0 – 6.2
slope	Slope of the peak exercise ST segment (0–2)	Categorical	0, 1, 2
ca	Number of major vessels colored by fluoroscopy (0–4)	Numeric	0 – 4
thal	Thalassemia type (3 = normal, 6 = fixed defect, 7 = reversible defect)	Categorical	3, 6, 7
target	Diagnosis of heart disease (0 = No, 1–4 = Disease presence)	Categorical	0 – 4

Need for the work:

Heart disease remains one of the most prevalent causes of death in the world, and the need for early, correct predictions cannot be overemphasized. Traditional diagnostic techniques are usually time-consuming and rely greatly on expert judgment; hence, they do not fit the bill as effective tools for timely intervention. The Kaggle Cleveland Heart Disease dataset includes various patient medical parameters which, if analyzed through machine learning algorithms, can easily reveal key patterns associated with the risk of a certain disease. However, due to its mix of numerical and categorical variables, its preprocessing and feature selection should be effectively performed. This research, therefore, addresses these challenges by developing an optimized machine learning framework using StandardScaler, SelectKBest, and PCA to enhance both accuracy and efficiency. The proposed approach aims at establishing a fast, reliable, and automated heart disease prediction model that supports healthcare professionals in making data-driven clinical decisions.

Model:

The proposed model is a **machine learning framework** designed to accurately predict the presence and severity of heart disease using the **Kaggle Heart Disease dataset**, which combines data from four sources—Cleveland, Hungary, Long Beach V, and Switzerland. The dataset contains **14 features and 303 records** representing patient health parameters.

The model follows a **pipeline architecture** consisting of three major components: **feature engineering**, **model training with cross-validation**, and **performance optimization**. During feature engineering, **StandardScaler** normalizes the features, **SelectKBest** selects the most relevant attributes using F-scores, and **Principal Component Analysis (PCA)** reduces dimensionality to minimize redundancy and multicollinearity.

A total of **six classifiers**—Logistic Regression, Naïve Bayes, Decision Tree, Random Forest, Gradient Boosting, and a Stacking Classifier—are implemented within a **pipeline grid**. **GridSearchCV** performs hyperparameter tuning with **5-fold cross-validation**, while **Joblib's parallel and delayed functions** enable parallel processing to accelerate model training and testing.

The data is split in an **80:20 ratio**, with additional internal splits (75:25) for validation, ensuring balanced class distribution and preventing overfitting. Model performance is evaluated using **Accuracy, Precision, Recall, F1-score**, and **processing time**.

Among all classifiers, **Gradient Boosting** achieves the **best performance** with **100% accuracy** on the test set, **98% validation accuracy**, and a **processing time of 0.003 ms**, outperforming other models in both speed and stability.

Coding:

#Importing packaged for EDA

import numpy as np

import pandas as pd

import matplotlib.pyplot as plt

import seaborn as sns

%matplotlib inline

import warnings

warnings.filterwarnings("ignore")

#Importing the CSV File

df=pd.read_csv("C:/Users/Shravani
Angalagunta/Downloads/Heart_disease_cleveland_new.csv")

#Display 5 rows of the dataset

df.head()

df.info()

df.isnull().sum()

#Descriptive Statistics

df.describe().T

df.columns

#Pair plot of Variables based on target

sns.pairplot(df,hue='target')

#Count of the number of patients with diseases vs with no diseases

sns.countplot(x='target', data = df)

#Pie chart showing the percentage of male patients vs female

df['sex'].value_counts().plot.pie(autopct='%.2f')

#Chest pain type based on Target variable

sns.barplot(y='age',x='cp',data=df,hue='target')

```

#Distribution of the age of the patients
sns.distplot(df['age'], kde=True)

#Correlation graph of all variables
corr=df.corr()
plt.figure(figsize= (15,15))
sns.heatmap(corr, linewidth=1,annot=True,linecolor='white')

#Defining the independent X variables and dependent y variables
X = df.drop('target', axis=1)
y = df['target']

#Train Test Split

from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3,
random_state=42)

#Feature Scaling

from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.fit_transform(X_test)

#Logistic Regression

from sklearn.linear_model import LogisticRegression
lr=LogisticRegression()
model_lr=lr.fit(X_train, y_train)
pred_lr = model_lr.predict(X_test)

from sklearn.metrics import confusion_matrix, accuracy_score,
precision_score, recall_score, f1_score
cm_lr=confusion_matrix(y_test,pred_lr)
print("Confusion Matrix  :", cm_lr)
accuracy_lr = accuracy_score(y_test, pred_lr)
print("Accuracy  :", accuracy_lr)
precision_lr = precision_score(y_test, pred_lr)

```

```

print("Precision :", precision_lr)
recall_lr = recall_score(y_test, pred_lr)
print("Recall   :", recall_lr)
F1_score_lr = f1_score(y_test, pred_lr)
print("F1-score :", F1_score_lr)

#Random Forest Classifier

from sklearn.ensemble import RandomForestClassifier

rf = RandomForestClassifier(n_estimators=300, random_state=42,
max_depth=5)
rf.fit(X_train, y_train)
pred_rf = rf.predict(X_test)
cm_rf=confusion_matrix(y_test,pred_rf)
print("Confusion Matrix   :", cm_rf)
accuracy_rf = accuracy_score(y_test, pred_rf)
print("Accuracy   :", accuracy_rf)
precision_rf = precision_score(y_test, pred_rf)
print("Precision :", precision_rf)
recall_rf = recall_score(y_test, pred_rf)
print("Recall   :", recall_rf)
F1_score_rf = f1_score(y_test, pred_rf)
print("F1-score :", F1_score_rf)

#Support Vector Machine(SVM)

from sklearn.svm import SVC

svc = SVC(C=2)
svc.fit(X_train, y_train)
pred_svm = svc.predict(X_test)
cm_svm=confusion_matrix(y_test,pred_svm)
print("Confusion Matrix   :", cm_svm)
accuracy_svm = accuracy_score(y_test, pred_svm)
print("Accuracy   :", accuracy_svm)

```



```
precision_svm = precision_score(y_test, pred_svm)
print("Precision :", precision_svm)
recall_svm = recall_score(y_test, pred_svm)
print("Recall   :", recall_svm)
F1_score_svm = f1_score(y_test, pred_svm)
print("F1-score :", F1_score_svm)
```

#KNN Method

```
from sklearn.neighbors import KNeighborsClassifier
knn = KNeighborsClassifier(n_neighbors=5)
knn.fit(X_train, y_train)
pred_knn = knn.predict(X_test)
cm_knn=confusion_matrix(y_test,pred_knn)
print("Confusion Matrix   :", cm_knn)
accuracy_knn = accuracy_score(y_test, pred_knn)
print("Accuracy   :", accuracy_knn)
precision_knn = precision_score(y_test, pred_knn)
print("Precision :", precision_knn)
recall_knn = recall_score(y_test, pred_knn)
print("Recall   :", recall_knn)
F1_score_knn= f1_score(y_test, pred_knn)
print("F1-score :", F1_score_knn)
```

#Extreme Gradient Boosting (XG Boost)

```
from xgboost import XGBClassifier
xg = XGBClassifier(learning_rate=0.05, n_estimators=100,
max_depth=15)
xg.fit(X_train, y_train)
pred_xg = xg.predict(X_test)
cm_xg=confusion_matrix(y_test,pred_xg)
print("Confusion Matrix   :", cm_xg)
accuracy_xg = accuracy_score(y_test, pred_xg)
```

```

print("Accuracy  :", accuracy_xg)
precision_xg = precision_score(y_test, pred_xg)
print("Precision :", precision_xg)
recall_xg = recall_score(y_test, pred_xg)
print("Recall   :", recall_xg)
F1_score_xg = f1_score(y_test, pred_xg)
print("F1-score :", F1_score_xg)
#XG Boost(Alternate method)
from sklearn.model_selection import RandomizedSearchCV
from xgboost import XGBClassifier
import numpy as np

xg = XGBClassifier(
    objective='binary:logistic',
    eval_metric='logloss',
    use_label_encoder=False,
    random_state=42
)

param_grid = {
    'n_estimators': [100, 200, 300, 400, 500],
    'learning_rate': [0.01, 0.05, 0.1, 0.2],
    'max_depth': [3, 5, 7, 10, 15],
    'min_child_weight': [1, 3, 5, 7],
    'gamma': [0, 0.1, 0.2, 0.3],
    'subsample': [0.6, 0.8, 1.0],
    'colsample_bytree': [0.6, 0.8, 1.0],
    'reg_lambda': [0.1, 1, 5],
    'reg_alpha': [0, 0.1, 0.5]

```

```
}
```

```
random_search = RandomizedSearchCV(  
    estimator=xg,  
    param_distributions=param_grid,  
    scoring='accuracy',  
    n_iter=50,  
    cv=5,  
    verbose=2,  
    n_jobs=-1,  
    random_state=42  
)
```

```
random_search.fit(X_train, y_train)  
print("Best Parameters:", random_search.best_params_)  
print("Best Accuracy:", random_search.best_score_)  
best_xg = random_search.best_estimator_  
y_pred = best_xg.predict(X_test)
```

```
from sklearn.metrics import accuracy_score, classification_report,  
confusion_matrix
```

```
print("Test Accuracy:", accuracy_score(y_test, y_pred))
```

```
print(classification_report(y_test, y_pred))
```

```
from xgboost import XGBClassifier
```

```
from sklearn.metrics import accuracy_score, classification_report
```

```
from sklearn.model_selection import train_test_split
```

```
from sklearn.preprocessing import StandardScaler
```

```
# Optional: scale data for stability
```

```
scaler = StandardScaler()
```

```
X_train_scaled = scaler.fit_transform(X_train)
```

```
X_test_scaled = scaler.transform(X_test)
```

```
# Optimized XGBoost model
```

```
xg = XGBClassifier(
```

```
learning_rate=0.03, # smaller step size → better generalization
```

```
n_estimators=600, # more trees since learning rate is smaller
```

```
max_depth=5, # avoids overfitting (15 was too large)
```

```
min_child_weight=3, # controls overfitting by minimum leaf weight
```

```
gamma=0.2, # regularization term for split decisions
```

```
subsample=0.8, # prevents overfitting by random row sampling
```

```
colsample_bytree=0.8, # random feature sampling per tree
```

```
reg_lambda=1, # L2 regularization
```

```
reg_alpha=0.1, # L1 regularization
```

```
objective='binary:logistic',
```

```
use_label_encoder=False,
```

```
eval_metric='logloss',
```

```
random_state=42,
```

```
n_jobs=-1
```

```
)
```

```
# Train the model
```

```
xg.fit(X_train_scaled, y_train)
```

```
# Predict and evaluate
```

```
pred_xg = xg.predict(X_test_scaled)
```

```
acc = accuracy_score(y_test, pred_xg)
```

```
print("Optimized XGBoost Accuracy:", acc)
```

```

print(classification_report(y_test, pred_xg))

#Random Forest Model

import seaborn as sns
import matplotlib.pyplot as plt

# Initialize Random Forest with tuned hyperparameters

rf = RandomForestClassifier(
n_estimators=300, # number of trees
max_depth=8, # prevent overfitting
min_samples_split=5, # min samples to split node
min_samples_leaf=2, # min samples per leaf
max_features='sqrt', # number of features per split
bootstrap=True, # bootstrap sampling
random_state=42,
n_jobs=-1 # parallel processing
)

# Train the model
rf.fit(X_train_scaled, y_train)

# Make predictions
y_pred = rf.predict(X_test_scaled)

# Evaluate model performance
accuracy = accuracy_score(y_test, y_pred)
print(f" Random Forest Accuracy: {accuracy * 100:.2f}%\n")
print("Classification Report:")
print(classification_report(y_test, y_pred))

# Confusion matrix

```

```

conf_matrixrf = confusion_matrix(y_test, y_pred)
sns.heatmap(conf_matrix, annot=True, fmt='d', cmap='Blues',
xticklabels=['No Disease', 'Disease'],
yticklabels=['No Disease', 'Disease'])
plt.title('Confusion Matrix - Random Forest')
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.show()

cm_xg_alternate = confusion_matrix(y_test, y_pred)
print("Confusion Matrix  :", cm_xg_alternate)
accuracy_xg_alternate = accuracy_score(y_test, y_pred)
print("Accuracy  :", accuracy_xg_alternate)
precision_xg_alternate = precision_score(y_test, y_pred)
print("Precision :", precision_xg_alternate)
recall_xg_alternate = recall_score(y_test, y_pred)
print("Recall  :", recall_xg_alternate)
F1_score_xg_alternate = f1_score(y_test, y_pred)
print("F1-score  :", F1_score_xg_alternate)

# Decision Tree Classifier for Heart Disease Prediction
from sklearn.tree import DecisionTreeClassifier, plot_tree
# Initialize Decision Tree with optimized parameters
dt = DecisionTreeClassifier(
criterion='entropy', # 'gini' or 'entropy'
max_depth=6, # limit tree depth to prevent overfitting
min_samples_split=5, # minimum samples per split
min_samples_leaf=2, # minimum samples per leaf node
splitter='best', # choose best split strategy
random_state=42
)

```

Train the model

```
dt.fit(X_train_scaled, y_train)
```

Make predictions

```
y_pred = dt.predict(X_test_scaled)
```

Evaluate model performance

```
accuracy = accuracy_score(y_test, y_pred)
```

```
print(f" Decision Tree Accuracy: {accuracy * 100:.2f}%\n")
```

```
print("Classification Report:")
```

```
print(classification_report(y_test, y_pred))
```

Confusion matrix visualization

```
conf_matrix_df = confusion_matrix(y_test, y_pred)
```

```
sns.heatmap(conf_matrix, annot=True, fmt='d', cmap='Greens',
```

```
xticklabels=['No Disease', 'Disease'],
```

```
yticklabels=['No Disease', 'Disease'])
```

```
plt.title('Confusion Matrix - Decision Tree')
```

```
plt.xlabel('Predicted')
```

```
plt.ylabel('Actual')
```

```
plt.show()
```

Optional: Visualize the Decision Tree structure (useful for report)

```
plt.figure(figsize=(15,8))
```

```
plot_tree(dt, filled=True, feature_names=X.columns, class_names=['No  
Disease', 'Disease'])
```

```
plt.title("Decision Tree Visualization")
```

```
plt.show()
```

```
conf_matrix_df = confusion_matrix(y_test, y_pred)
```

```
print("Confusion Matrix  :", conf_matrix_df)
accuracy_df = accuracy_score(y_test,y_pred)
print("Accuracy  :", accuracy_df)
precision_df = precision_score(y_test, y_pred)
print("Precision :", precision_df)
recall_df = recall_score(y_test, y_pred)
print("Recall  :", recall_df)
F1_score_df = f1_score(y_test, y_pred)
print("F1-score  :", F1_score_df)

#logistic regression
result1 = ["1.,"Logistic Regression"]
result1.append(round(accuracy_lr*100,2))
result1.append(round(precision_lr*100,2))
result1.append(round(recall_lr*100,2))
result1.append(round(F1_score_lr*100,2))

#random forest classifier
result2 = ["2.,"Random Forest Classifier"]
result2.append(round(accuracy_rf*100,2))
result2.append(round(precision_rf*100,2))
result2.append(round(recall_rf*100,2))
result2.append(round(F1_score_rf*100,2))

#support vector machine
result3 = ["3.,"Support Vector Machine"]
result3.append(round(accuracy_svm*100,2))
result3.append(round(precision_svm*100,2))
result3.append(round(recall_svm*100,2))
result3.append(round(F1_score_svm*100,2))
```


#knn

```
result4 = ["4.", "K Nearest Neighbours"]  
result4.append(round(accuracy_knn*100,2))  
result4.append(round(precision_knn*100,2))  
result4.append(round(recall_knn*100,2))  
result4.append(round(F1_score_knn*100,2))
```

#xg boost

```
result5 = ["5.", "XG Boost"]  
result5.append(round(accuracy_xg*100,2))  
result5.append(round(precision_xg*100,2))  
result5.append(round(recall_xg*100,2))  
result5.append(round(F1_score_xg*100,2))
```

#xg boost alternate method

```
result6 = ["6.", "XG Boost Alternate method"]  
result6.append(round(accuracy_xg_alternate*100,2))  
result6.append(round(precision_xg_alternate*100,2))  
result6.append(round(recall_xg_alternate*100,2))  
result6.append(round(F1_score_xg_alternate*100,2))
```

#Decision Tree method

```
result7 = ["7.", "Decision Tree method"]  
result7.append(round(accuracy_df*100,2))  
result7.append(round(precision_df*100,2))  
result7.append(round(recall_df*100,2))  
result7.append(round(F1_score_df*100,2))
```

```
from prettytable import PrettyTable

Result_table =
PrettyTable(["S.No.", "Model", "Accuracy", "Precision", "Recall", "F1 Score"])

Result_table.add_row(result1)
Result_table.add_row(result2)
Result_table.add_row(result3)
Result_table.add_row(result4)
Result_table.add_row(result5)
Result_table.add_row(result6)
Result_table.add_row(result7)
print(Result_table)
```

Explanation:

This code carries out a full analysis on the dataset for heart disease prediction using several machine learning algorithms. It starts with data exploration and visualization to understand patterns in the dataset, such as age distribution, gender ratio, and correlations between features. After cleaning and scaling the data, it splits the dataset into training and testing sets. Then, it trains and evaluates seven models-Logistic Regression, Random Forest, Support Vector Machine (SVM), K-Nearest Neighbours (KNN), XGBoost, Tuned XGBoost (Alternate), and Decision Tree- calculating key metrics like Accuracy, Precision, Recall, and F1-Score for each. Finally, all the model performances are neatly summarized in a comparison table that will help identify the most accurate and reliable model for predicting heart disease.

Performance Analysis:

The performance analysis of various machine learning models in predicting heart disease indicates that XGBoost and its alternative, along with the Decision Tree method, are the top performers among all the models. The accuracy is 90.16%, precision is 86.67%, recall is 92.86%, and the F1-score is 89.66%. This demonstrates a very strong predictive power and a great balance between the ability to recall true positive cases and precision in terms of preventing false positives. Logistic Regression also stands out because it does really well at 87.91% for accuracy, hence reliable for depicting linear relationships within the dataset. Contrariwise, Random Forest yielded a relatively lower accuracy of 85.71%, whereas the Support Vector Machine (SVM) and K-Nearest Neighbours (KNN) recorded 82.42%, showing fairly weak generalization on this dataset. Overall, the results indicate that, due to their capabilities in modeling nonlinear interactions of features and handling complex data patterns, the ensemble, and tree-based models like XGBoost and Decision Tree are more adept for heart disease prediction.

References:

1. <https://www.kaggle.com/datasets/aavigan/cleveland-clinic-heart-disease-dataset/data>
2. Optimized Feature Engineering and Machine Learning Methods for Heart Disease Prediction from 2024 IEEE 6th International Conference on Cybernetics, Cognition and Machine Learning Applications
3. Study of Heart Disease Prediction Using Machine Learning and Data Mining Techniques Prof. Anuradha S. Deokar¹ , Rudraksh.R. Kokane² , Aditya A. Bodake³ , Tanmay. S. Bhagwat⁴ , Devraj M. Malawe⁵ from International Journal for Research in Applied Science & Engineering Technology (IJRASET)

4. M. Qadri, A. Raza, K. Munir, and M. Almutairi, "Effective Feature Engineering Technique for Heart Disease Prediction with Machine Learning," vol. 11, p. 56214, Jan. 2023, DoI: 10.1109/access.2023.3281484
5. S. Ouyang, "Research of Heart Disease Prediction Based on Machine Learning," Apr. 2022, doi: 10.1109/aemcse55572.2022.00071.
6. A. Jain, K. Kumar, R. G. Tiwari, N. B. Jain, V. Gautam, and N. K. Trivedi, "Machine Learning-Based Detection of Cardiovascular Disease using Classification and Feature Selection," Apr. 2023, doi: 10.1109/csnt57126.2023.10134672
7. F. Tasnim and S. U. Habiba, "A Comparative Study on Heart Disease Prediction Using Data Mining Techniques and Feature Selection," Jan. 2021, doi: 10.1109/icrest51555.2021.9331158.
8. Yuan, X., Chen, J., Zhang, K., Wu, Y., & Yang, T. (2022). A stable AI Based binary and multiple class heart disease prediction model for IOMT. IEEE Transactions on Industrial Informatics, 18(3), 2032–2040
9. Pasha, Syed Javeed, and Elhassan Mohamed. "Novel Feature Reduction (NFR) Model With Machine Learning and Data Mining Algorithms for Effective Disease Risk Prediction." IEEE Access 8 (January 1, 2020): 184087–108.
10. Bashir, S., Almazroi, A. A., Ashfaq, S., Almazroi, A. A., & Khan, F. H. (2021). A Knowledge-Based clinical decision support system utilizing an intelligent ensemble voting scheme for improved cardiovascular disease prediction. IEEE Access, 9, 130805–130822.